

Lab. 3
SQL CRUD & Querying

Content

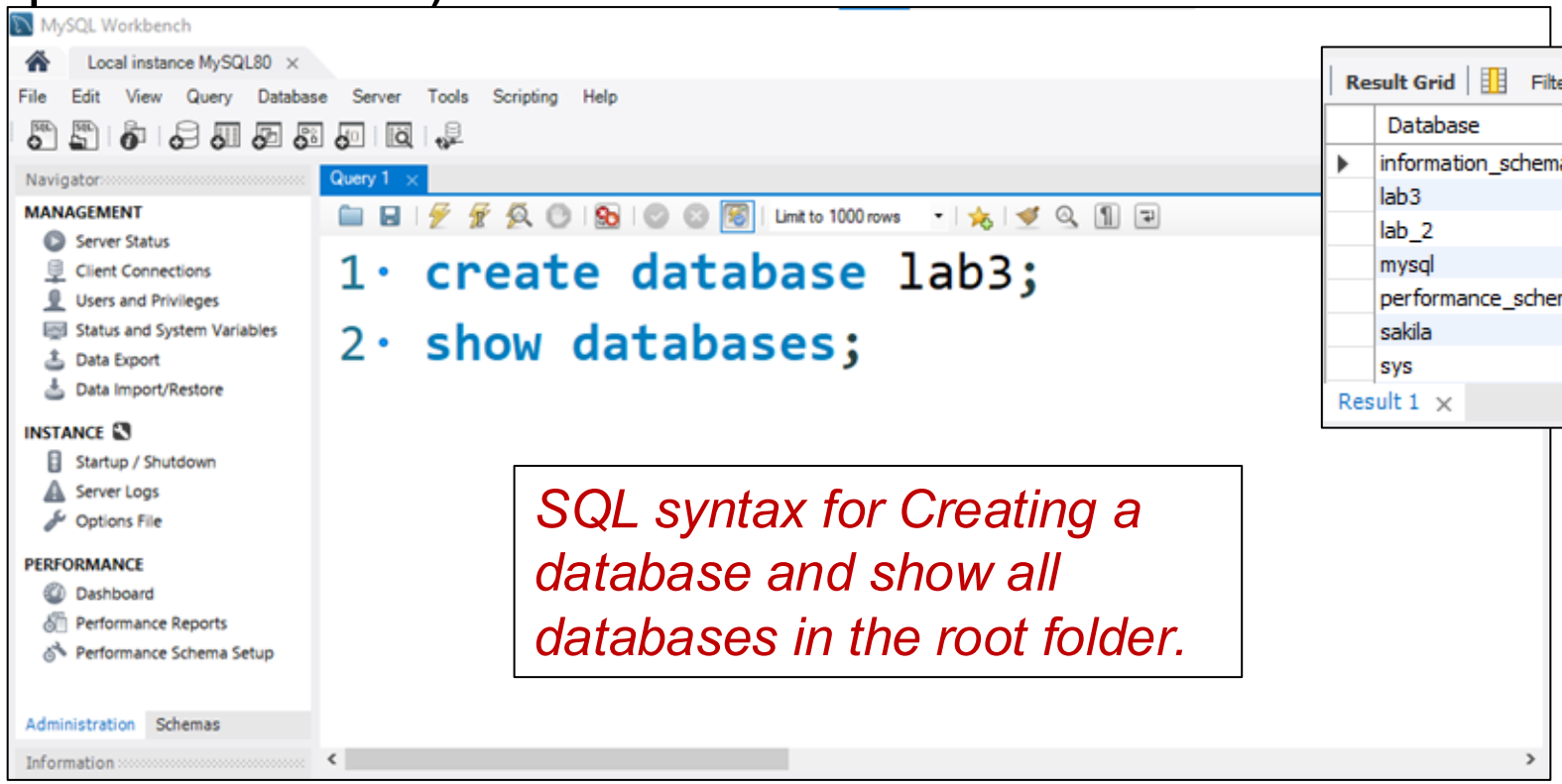
1. Recap: from an empty schema to live data
2. Create a database
3. INSERT and filling the database with seed data
4. SELECT, DISTINCT, WHERE: reading and filtering
5. Pattern matching, IN, BETWEEN, ORDER BY, LIMIT
6. Aggregate functions; GROUP BY and HAVING
7. MySQL mathematical functions
8. More about MySQL
9. UPDATE and DELETE: changing and removing, safely
10. Primary Key & Foreign key with Alter table
11. MySQL table joins
12. MySQL database column types
13. Good Habits with CRUD
14. where we're going next

Recap: from an empty schema to live data

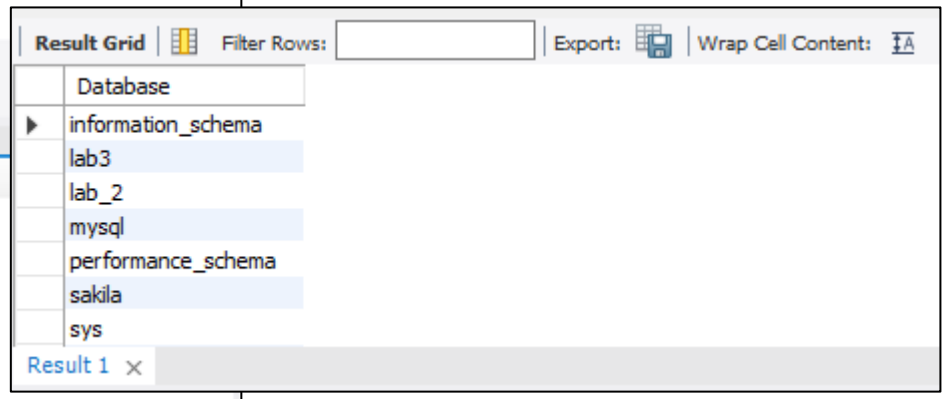
- In Lab 2 you designed a schema and built the empty tables in MySQL Workbench.
- The tables exist but hold no rows yet, a blueprint with no data.
- Today we bring it to life: add data, read it back, change it, and remove it.
- Everything runs in the Workbench query tab; we connect Node.js to this database in Lab 4.

Create a Database

Open the MySQL Workbench, and Start Querying (local instance & password root)



SQL syntax for Creating a database and show all databases in the root folder.



Ctrl + Shift + Enter to execute the query or Query ➡ Execute (All or Selection)

Create a Database (Contd.)

CRUD: The Four Things We Do With Data

- ✓ Almost everything we do to stored data is one of four operations together, CRUD:
- ✓ Create — add new rows → INSERT
- ✓ Read — fetch rows back → SELECT
- ✓ Update — change existing rows → UPDATE
- ✓ Delete — remove rows → DELETE
- ✓ Type each as a SQL statement in the query tab and read the outcome in the results grid.

Create a Database (Contd.)

Our Tables for Today

We use three tables from your Lab 2 design: students, courses, and the enrolments that link them.

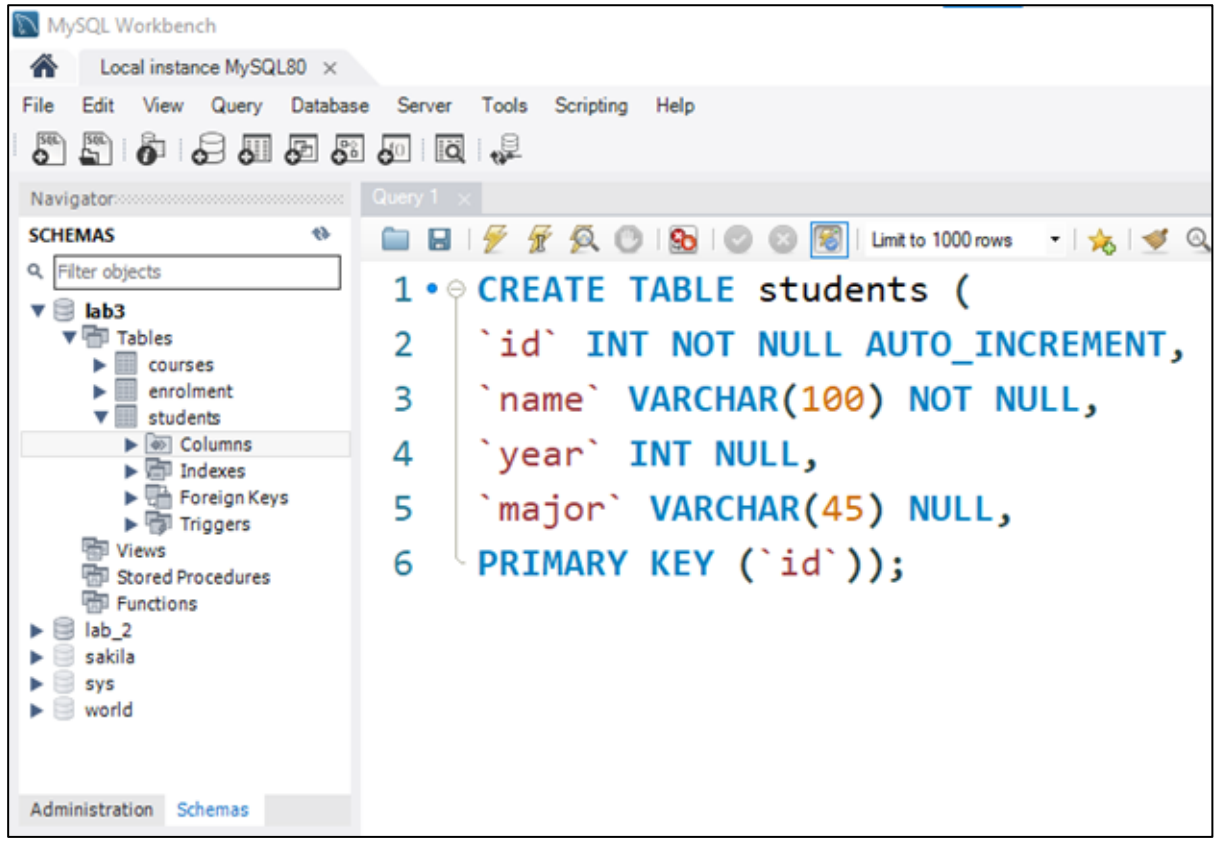
```
students( id, name, major, year )
courses( id, title, credit )
enrolment( id, student_id, course_id, grade )

student_id  -> students(id)
course_id   -> courses(id)
```

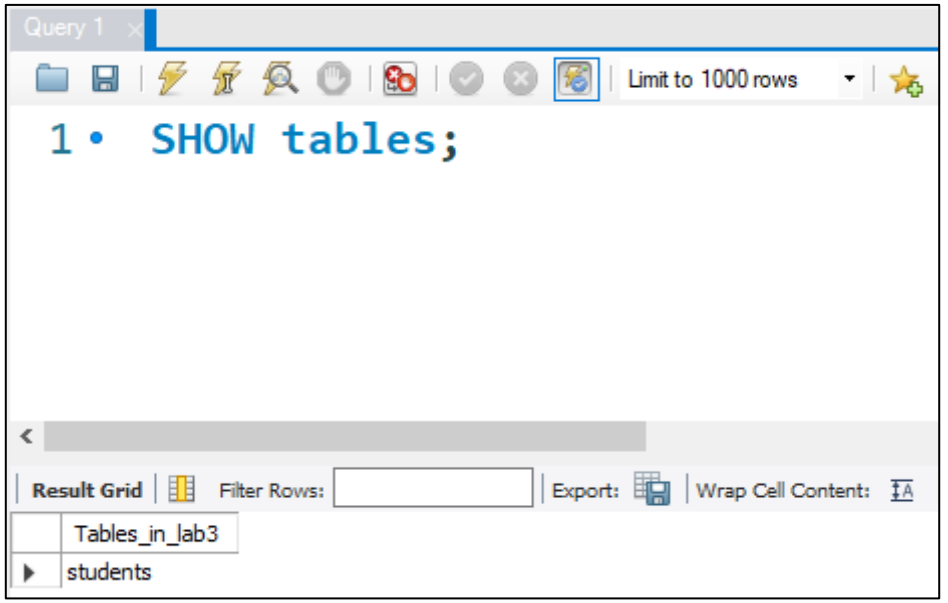
****enrolment is the junction table: each row links one student to one course.*

Create a Database (Contd.)

Command for using a database & creating a table

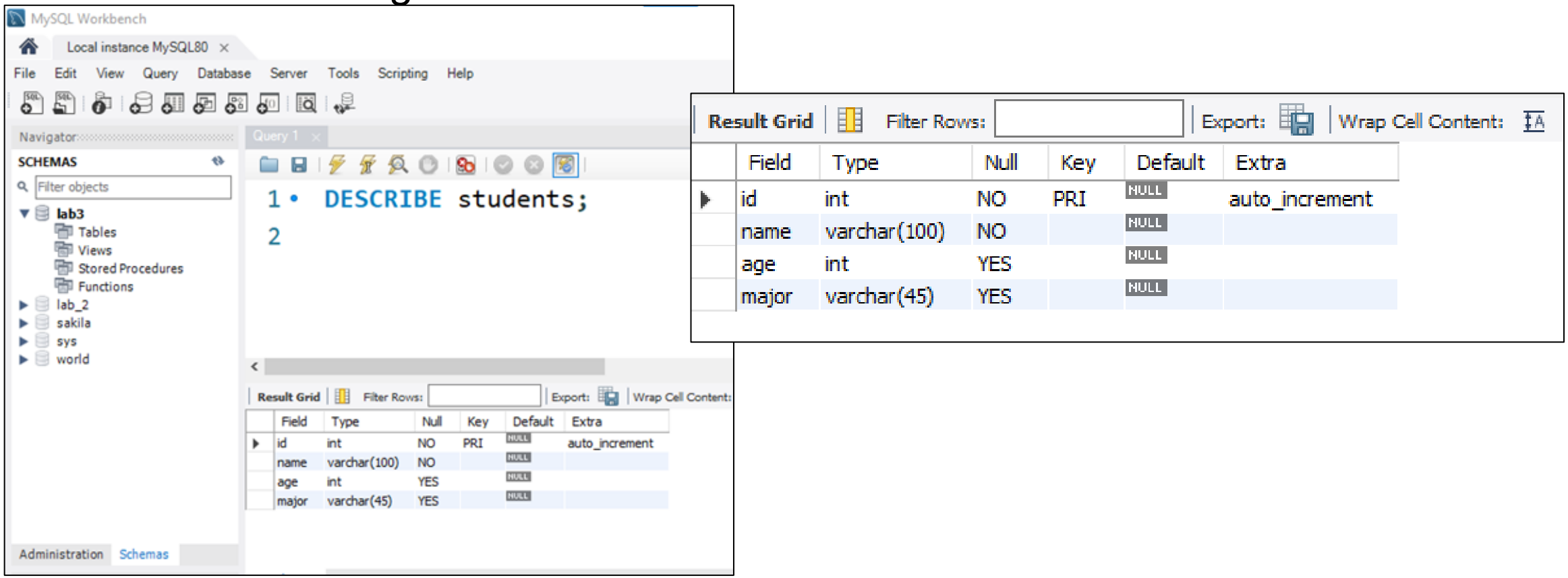


Ctrl + Shift + Enter to execute the query & SHOW tables to see the created table.



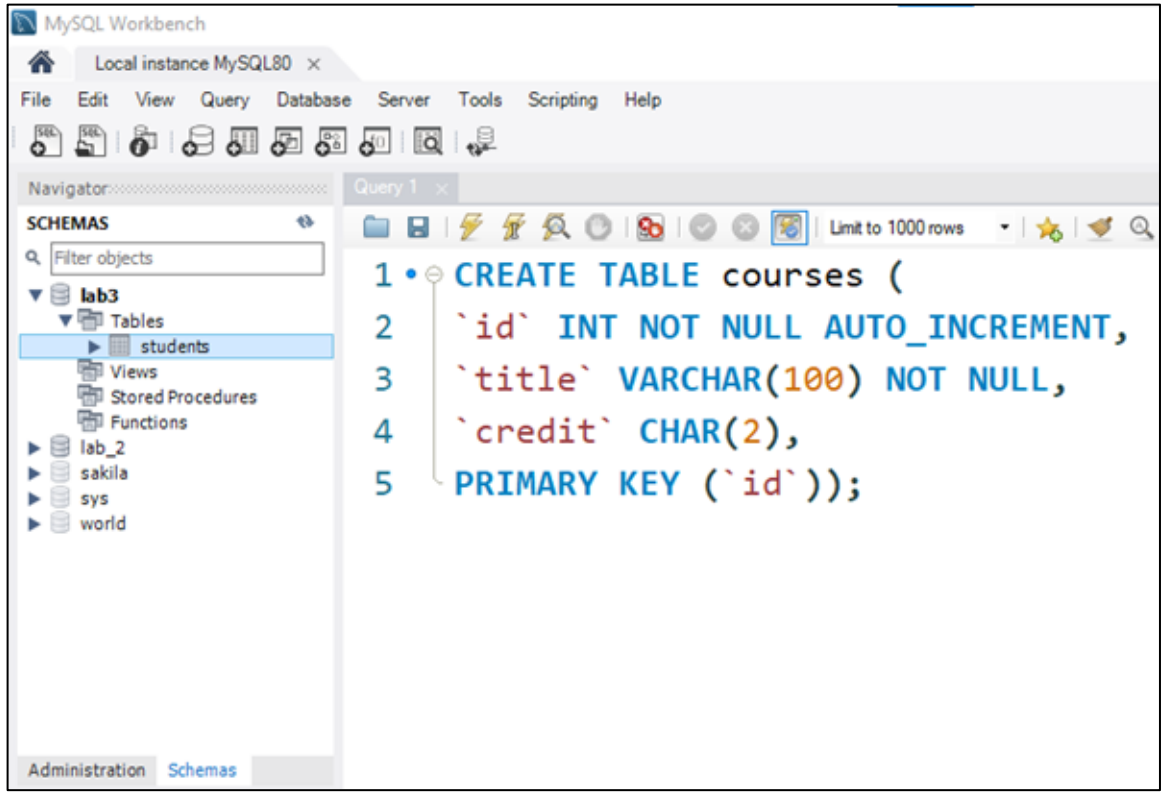
Create a Database (Contd.)

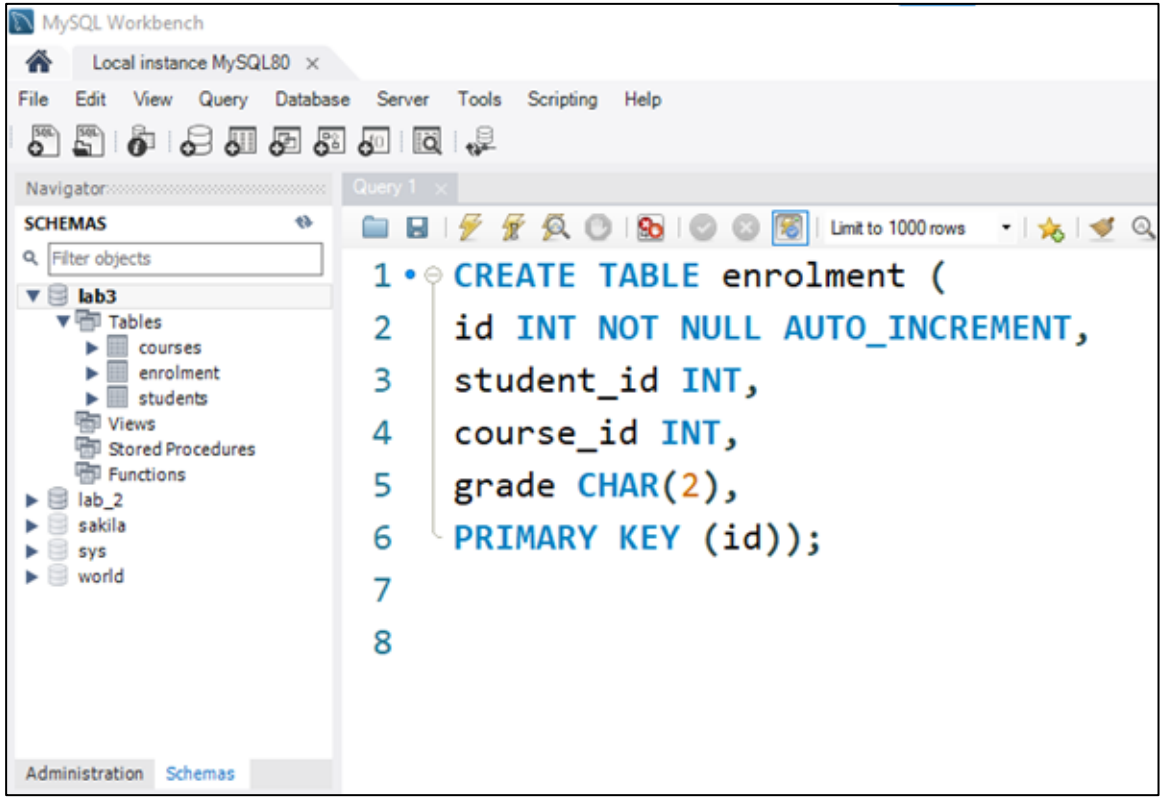
Command for seeing the structure of the table



Create a Database (Contd.)

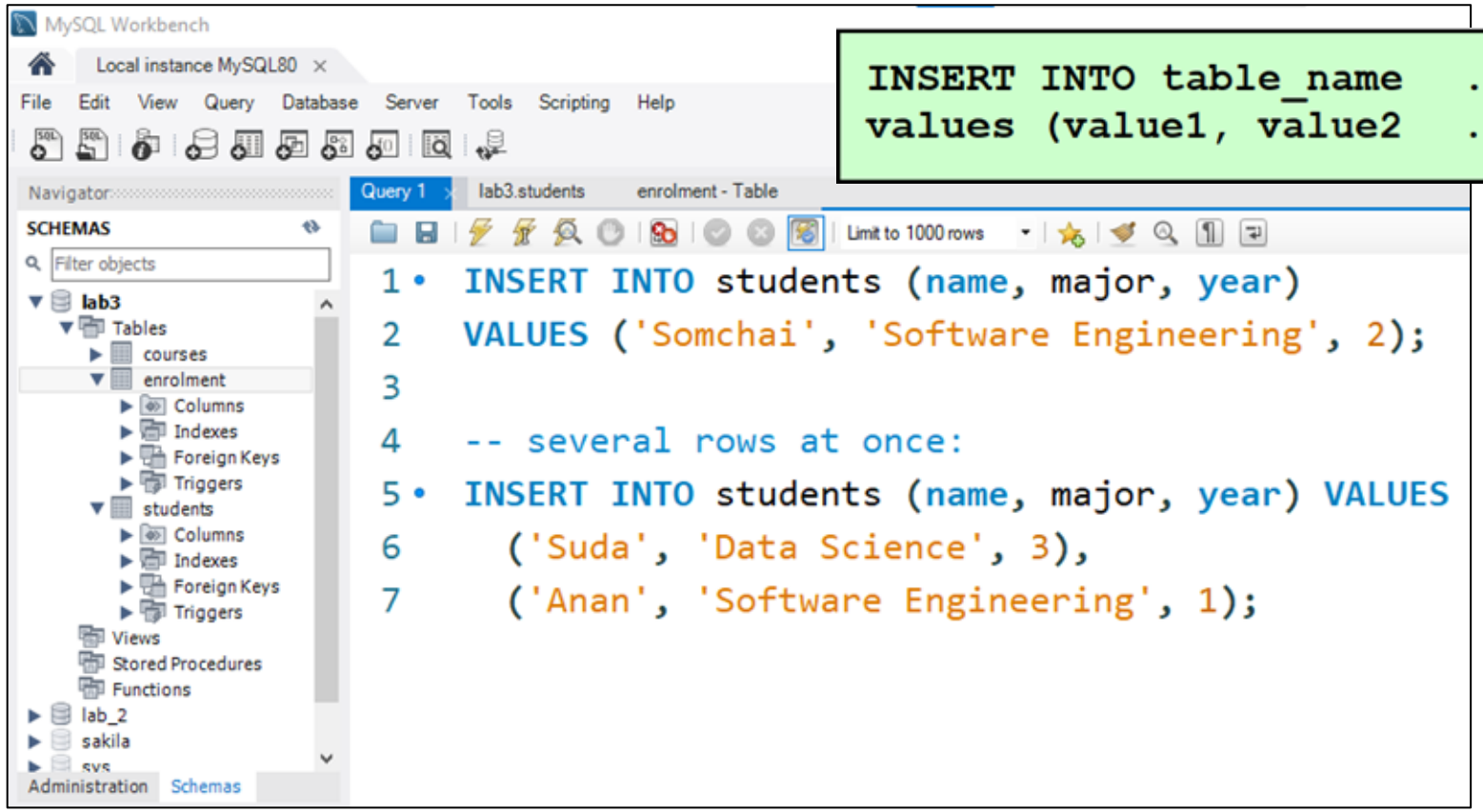
Our Tables for Today





INSERT and Filling the Database with Seed Data

Command for inserting data into tables




```
INSERT INTO table_name   ....(column1,column2 )
values (value1, value2   ...)
```

We omit id, AUTO_INCREMENT fills it. Text goes in quotes; numbers do not.

INSERT and Filling the Database with Seed Data (Contd.)

Before we can read or summarize anything, the tables need data. Run these once, in order.

```
INSERT INTO students (name, major, year) VALUES
('Somchai','Software Engineering',2),
('Suda','Data Science',3),
 ('Anan','Software Engineering',1),
('Nicha','Information Technology',4),
('Krit','Data Science',2),
('Ploy','Software Engineering',3),
('Tanya','Information Technology',1),
('Wichai','Data Science',4);
```

INSERT and Filling the Database with Seed Data (Contd.)

Before we can read or summarize anything, the tables need data. Run these once, in order.

```
INSERT INTO courses (title, credit) VALUES
  ('Database Programming',3),('Web
Development',3),('Algorithms',4),
('Statistics',3),('Operating Systems',4);
```

*Students and courses first, and then enrolments. Each enrolment refers to ids that must already exist. ***We are going to talk about this later.*

```
INSERT INTO enrolment (student_id, course_id, grade) VALUES
  (1,1,'A'),(1,2,'B+'),(2,1,'A'),(2,4,'B'),(3,2,'C+'),(4,3,'B+'),
(4,5,'A'),(5,1,'B'),(5,4,'A'),(6,2,'B+'),(6,3,'C'),(7,5,'B'),(8,1,'A'),(8,4,'A');
```

INSERT and Filling the Database with Seed Data (Contd.)

Lab Exercise 1: **Fill the database**

- Run the three seed INSERTs previously, in order, on your empty tables.
- Add at least two more students of your own with one multi-row INSERT.
- Let AUTO_INCREMENT assign every id (do not type it).
- Run SELECT * FROM students; you should see ten rows in the grid.

Starter code:

```
INSERT INTO students (name, major, year) VALUES ('____','____',__),('____','____',__);
```

SELECT, DISTINCT, WHERE: Reading and Filtering

SELECT: Reading Data

SELECT reads rows back. Name the columns you want, or use * for all of them.

```
SELECT name, major FROM students;

SELECT name AS student, year AS study_year FROM students;

-- DISTINCT collapses repeats: one row per major
SELECT DISTINCT major FROM students;
```

SELECT, DISTINCT, WHERE: Reading and Filtering (Contd.)

WHERE: Filtering Rows

WHERE keeps only the rows that match a condition.

```
SELECT * FROM students WHERE major = 'Data Science';

SELECT name FROM students WHERE year >= 3;

SELECT * FROM students
WHERE major = 'Software Engineering' AND year = 1;
```

Compare with =, <, >, <=, >=, <>. Combine conditions with AND / OR / NOT and parentheses.

Pattern matching, IN, BETWEEN, ORDER BY, LIMIT

Three handy ways to write a WHERE condition beyond simple comparison.

```
-- LIKE: % is any run of characters
SELECT name FROM students WHERE name LIKE 'S%';
SELECT name FROM students WHERE name NOT LIKE '%a%';

-- IN: match any value from a list (Logical or)
SELECT * FROM students
WHERE major IN ('Data Science', 'Information Technology');

-- BETWEEN: an inclusive range (A ≤ value ≤ B)
SELECT name, year FROM students WHERE year BETWEEN 2 AND 3;
```

NOT inverts each one:
NOT LIKE, NOT IN, NOT BETWEEN.

Pattern matching, IN, BETWEEN, ORDER BY, LIMIT (Contd.)

ORDER BY sorts the result; LIMIT caps how many rows come back.

```
SELECT name, year FROM students ORDER BY year;

SELECT name, year FROM students ORDER BY year
DESC;

SELECT * FROM students ORDER BY name LIMIT 3;

SELECT * FROM students ORDER BY name LIMIT 1, 2;
```

Order is ascending by default; add DESC for descending. LIMIT is handy for 'top N' lists.

Pattern matching, IN, BETWEEN, ORDER BY, LIMIT (Contd.)

Lab Exercise 2: ***Query your data***

- List every student's name and major; then only the students in one major.
- Use BETWEEN to show students in year 2 or 3, sorted by name.
- Use LIKE to list students whose name contains the letter 'a'.
- Return just the first three rows of any query with LIMIT.

Starter code:

```
SELECT ____ FROM students
WHERE ____
ORDER BY ____ LIMIT 3;
```

Aggregate functions; GROUP BY and HAVING

Aggregate functions condense many rows into a single value. Name the result with AS.

```
SELECT COUNT(*) AS num_students FROM students;

SELECT AVG(year) AS avg_year FROM students;

SELECT MIN(credit) AS min_cr,
       MAX(credit) AS max_cr,
       SUM(credit) AS total_cr
FROM courses;
```

The five: COUNT, SUM, AVG, MIN, MAX. COUNT(*) counts rows; COUNT(col) counts non-NULL values.

Aggregate functions; GROUP BY and HAVING (Contd.)

GROUP BY and HAVING: GROUP BY runs an aggregate once per group; HAVING filters those groups.

```
-- one row per major, with a count
SELECT major, COUNT(*) AS num_students
FROM students
GROUP BY major;

-- keep only the larger majors
SELECT major, COUNT(*) AS num_students
FROM students
GROUP BY major
HAVING COUNT(*) >= 3;
```

*WHERE (*looks at raw, individual rows: one student at a time*) filters rows before grouping; HAVING (*looks at the groups themselves*) filters groups after. Aggregates can't go in WHERE.*

Aggregate functions; GROUP BY and HAVING (Contd.)

Lab Exercise 3: ***Summarize your data***

- ❑ Count how many students there are in total.
- ❑ Use GROUP BY to count the students in each major.
- ❑ Add HAVING to show only majors with three or more students.
- ❑ Order that result with the most populous major first.

Starter code:

```
SELECT major, COUNT(*) AS n
FROM students
GROUP BY major
HAVING ____ ORDER BY ____;
```

MySQL Mathematical Functions

1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Modulus (% and MOD (x,y))
6. Absolute (ABS (x))
7. Sign (SIGN (x))
8. Power (POWER (x,y))
9. Sqrt (SQRT (x))
10. Round (ROUND(x), ROUND (x,y))
11. Tan, sin and cos (TAN (x), SIN (x), COS (x))

```
        SELECT 87 % 9
+--------+
|87%   9 |
+--------+
|       6 |
+--------+
1 row in set (0.00 sec)
```

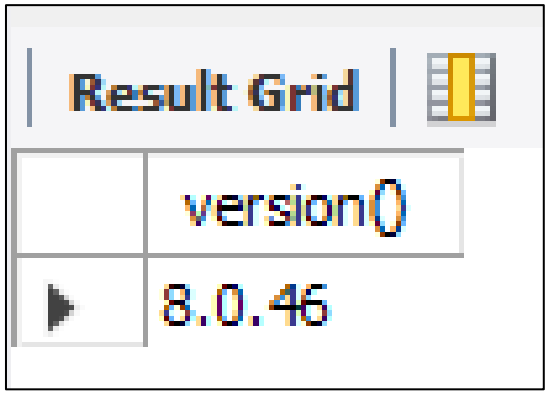
```
        SELECT POWER(4,3)
+-------------+
| POWER(4,3)  |
+-------------+
|   64.000000|
+-------------+
1 row in set (0.00 sec)
```

```
      SELECT SIN(0)
+----------+
| SIN(0)   |
+----------+
| 0.000000 |
+----------+
1  row in set (0.00 sec)
```

More about MySQL

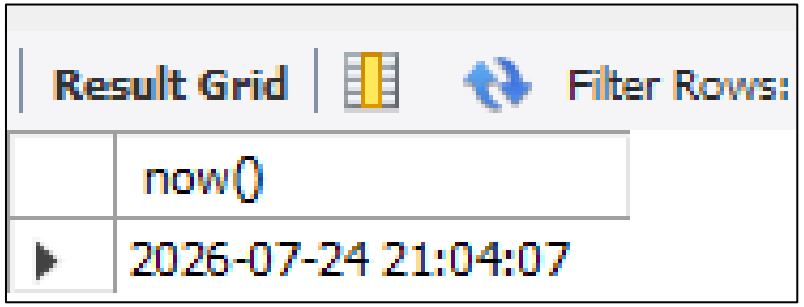
Check the version

```
SELECT version()
```



Check the current time

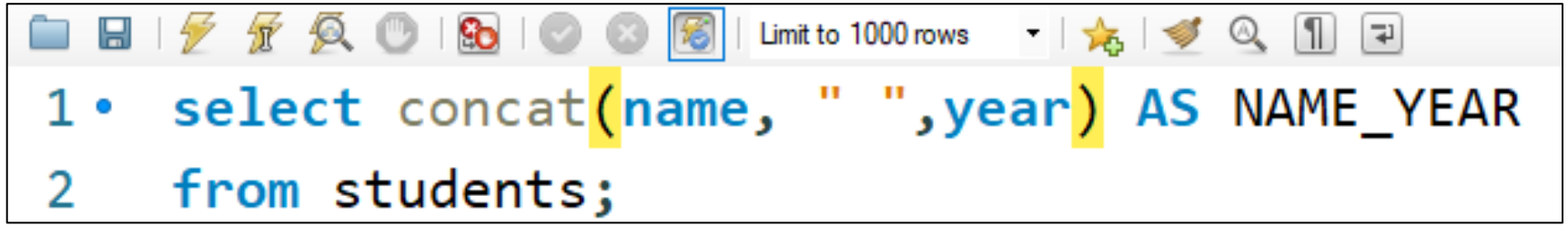
```
SELECT now()
```

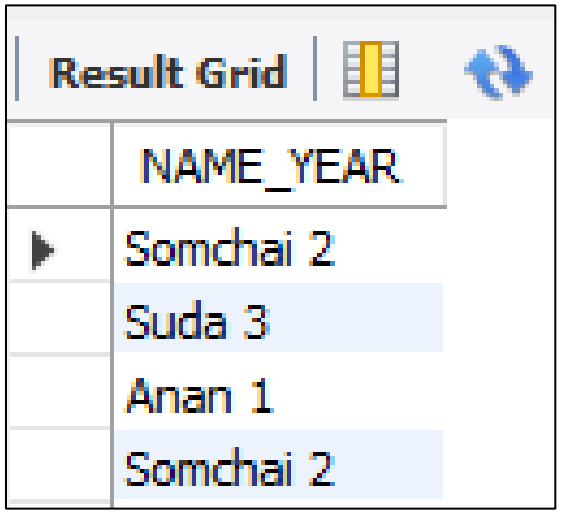


* You can select year, month and day similar to above.

More about MySQL (Contd.)

Concatenation





CONCAT() function joins two or more expressions or column values into a single string

UPDATE and DELETE: changing and removing, safely

UPDATE changes columns in the rows that match a WHERE condition.

```
UPDATE table_name
  SET column_name1 = value1,
      column_name2 = value2, ...
[WHERE conditions];
```

```
UPDATE students SET major = 'ICT'
WHERE id = 2;

UPDATE students SET year = year + 1
WHERE year < 4;
```

Always include WHERE without it, every row changes. Run the same WHERE as a SELECT first to check.

UPDATE and DELETE: changing and removing, safely (Contd.)

DELETE removes whole rows that match a WHERE condition.

```
DELETE FROM enrolment WHERE
student_id = 5;

DELETE FROM students WHERE id = 5;
```

DELETE FROM students; with no WHERE empties the table. Like UPDATE, preview with a SELECT first.

UPDATE and DELETE: changing and removing, safely (Contd.)

Dropping tables (*for your knowledge only)

```
 DELETE from employee_data
Query OK, 0 rows affected (0.00 sec)
```

```
 SHOW TABLES
+---------------------+
| Tables in employees |
+---------------------+
| employee_data       |
+---------------------+
1  rows in set (0.00 sec)
```

```
 DROP TABLE employee_data
Query OK, 0 rows affected (0.01 sec)
```

Aggregate functions; GROUP BY and HAVING (Contd.)

Lab Exercise 4: ***Change and remove data***

- ❑ Update one student's major to a different value.
- ❑ Increase the year of every first-year student by one.
- ❑ Delete one student you added by mistake.
- ❑ Before each UPDATE/DELETE, run its WHERE as a SELECT to confirm the rows.

Starter code:

```
UPDATE students SET ____ = ____ WHERE ____;
DELETE FROM students WHERE ____;
```

Primary Key & Foreign key with Alter Table

Creating index

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

```
CREATE UNIQUE INDEX index_name ON table_name (column1, column2, ...);
```

```
create INDEX ind_ID ON enrolment(student_id);
```

Before

Column	Type	Nullable	Indexes
id	int	NO	PRIMARY
student_id	int	YES	
course_id	int	YES	
grade	char(2)	YES	

After

Column	Type	Nullable	Indexes
id	int	NO	PRIMARY
student_id	int	YES	ind_ID
course_id	int	YES	
grade	char(2)	YES	

Primary Key & Foreign key with Alter Table (Contd.)

Alter Table

Alter Table TABLE_NAME modify column COLUMN_NAME DATA_TYPE;

Alter Table students modify column Id bigint;

ALTER TABLE statement is a versatile command used to modify the structure of an existing database table.

Before

Info | Columns | Indexes | Triggers | Foreign keys | Partitions | Grants | DDL

Column	Type	Default Value	Nullable
id	int		NO
major	varchar(45)		YES
name	varchar(100)		NO
year	int		YES

After

Info | Columns | Indexes | Triggers | Foreign keys | Partitions | Grants | DDL

Column	Type	Default Value	Nullable
Id	bigint		NO
major	varchar(45)		YES
name	varchar(100)		NO
year	int		YES

Primary Key & Foreign key with Alter Table (Contd.)

Quick MySQL ALTER TABLE options guide

Add Column: ADD c1 INT;

Remove Column: DROP c1;

Change Type: MODIFY c1 VARCHAR(50);

Rename Column: RENAME COLUMN c1 TO c2;

Rename Table: RENAME TO t2;

Add Primary Key: ADD PRIMARY KEY (id);

Add Foreign Key: ADD CONSTRAINT fk FOREIGN KEY (c1) REFERENCES t2(id);

Add Index: ADD INDEX idx (c1);

Drop Index/Constraint: DROP INDEX idx; / DROP FOREIGN KEY fk;

You need to mention "ALTER TABLE TABLE_NAME" before the structural change.

Primary Key & Foreign key with Alter Table (Contd.)

Adding Foreign Keys: ALTER TABLE

Enrolment already exists with student_id and course_id as plain columns. ALTER TABLE turns each one into a real foreign key, linking it back to the table it belongs to and once it's in place, MySQL enforces the relationship for you.

```
- - students, courses and enrolments already exist, link them now.
ALTER TABLE enrolment ADD CONSTRAINT FK_EnrolmentStudent FOREIGN
KEY (student_id) REFERENCES students(id);

ALTER TABLE enrolment ADD CONSTRAINT FK_EnrolmentCourse FOREIGN
KEY (course_id) REFERENCES courses(id);
```

In Lab 2 you built this same relationship visually in Workbench, this is what it looks like as SQL. Once the constraint exists, MySQL rejects any enrolment for a student_id or course_id that doesn't actually exist.

Primary Key & Foreign key with Alter Table (Contd.)

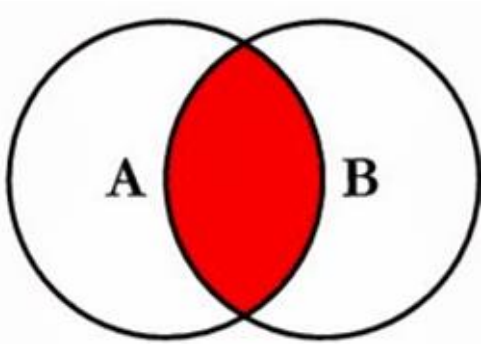
Foreign Keys in Action

Try enrolling a student that doesn't exist, MySQL stops you before bad data gets in.

```
INSERT INTO enrolments (student_id, course_id, grade)
VALUES (999, 1, 'A');
-- Error 1452: Cannot add or update a child row:
-- a foreign key constraint fails
```

This is the payoff of the constraint: without it, that row would have been inserted silently, and you'd have an enrolment pointing at a student who was never there.

MySQL Table Joins



Reading Across Tables: the JOIN

- INNER JOIN - selects records that have matching values in both tables
- Your data is split across tables on purpose: students, courses, and the enrolments that link them.
- To answer 'which courses is each student taking?' we must combine tables.
- A JOIN matches rows from two tables using the foreign key, enrolments.student_id matches students.id.
- An INNER JOIN returns only the rows that have a match on both sides.

MySQL Table Joins (Contd.)

Join the tables on the columns that link them, then pick the columns you want.

```
SELECT s.name, c.title
FROM enrolment e
JOIN students s ON e.student_id = s.id
JOIN courses  c ON e.course_id  = c.id;

-- CONCAT builds one tidy column
SELECT CONCAT(s.name, ' — ', c.title) AS enrolment
FROM enrolment e
JOIN students s ON e.student_id = s.id
JOIN courses  c ON e.course_id  = c.id
ORDER BY s.name;
```

Aliases (e, s, c) keep it short. Each ON says which columns match; add WHERE / ORDER BY / GROUP BY as before.

MySQL Table Joins (Contd.)

Lab Exercise 5: ***Join the tables***

- ❑ Write a JOIN that lists each student's name next to the course titles they take.
- ❑ Add a WHERE to show the enrolments for just one student.
- ❑ Sort the result by student name.
- ❑ Challenge: with GROUP BY, count how many courses each student takes.

Starter code:

```
SELECT s.name, c.title
FROM enrolment e
JOIN students s ON e.student_id = s.id
JOIN courses  c ON e.course_id  = c.id;
```

MySQL database column types

1. Integer
2. Date
3. Text

Numeric types

- **TINYINT**: very small numbers; suitable for ages. Actually, we should have used this data type for employee ages and number of children. Can store numbers between 0 to 255 if UNSIGNED clause is applied, else the range is between -128 to 127.
- **SMALLINT**: Suitable for numbers between 0 to 65535 (UNSIGNED) or -32768 to 32767.
- **MEDIUMINT**: 0 to 16777215 with UNSIGNED clause or -8388608 to 8388607.
- **INT**: UNSIGNED integers fall between 0 to 4294967295 or -2147683648 to 2147683647.
- **BIGINT**: Huge numbers. (-9223372036854775808 to 9223372036854775807)
- **FLOAT**: Floating point numbers (single precision)
- **DOUBLE**: Floating point numbers (double precision)
- **DECIMAL**:Floating point numbers represented as strings.

MySQL database column types (Contd.)

Date types

- **DATE**: YYYY-MM-DD (Four digit year followed by two digit month and date)
- **TIME**: hh:mm:ss (Hours:Minutes:Seconds)
- **DATETIME**: YYYY-MM-DD hh:mm:ss (Date and time separated by a space character)
- **TIMESTAMP**: YYYYMMDDhhmmss
- **YEAR**: YYYY (4 digit year)

MySQL database column types (Contd.)

Text types

- **CHAR(x)**: where x can range from 1 to 255.
- **VARCHAR(x)**: x ranges from 1 - 255
- **TINYTEXT**: small text, case insensitive
- **TEXT**: slightly longer text, case insensitive
- **MEDIUMTEXT**: medium size text, case insensitive
- **LONGTEXT**: really long text, case insensitive

Good Habits with CRUD

- Read before you write: run the WHERE as a SELECT first, so you see exactly which rows an UPDATE or DELETE will touch.
- Never run UPDATE or DELETE without a WHERE unless you truly mean every row.
- Change a little at a time, then re-check with a SELECT.
- Use consistent values (the same spelling of a major everywhere) so filters and joins work reliably.

Where We're Going Next

- You can now create and fill tables, then read, filter, sort and summarize data with the SQL you have met today.
- You can change and remove rows safely, alter a table's structure and keys, and join related tables together.
- In Lab 4 we connect this same database to a Node.js program with the mysql2 library running your SQL from code.
- Before then, make sure you are comfortable with SELECT, WHERE, GROUP BY and JOIN.

Thank you & Assignment time…